Graph Embedding-based Fraud Detection on the Ethereum Blockchain

Jakob Wankmüller*
M. Sc. Data Science
Ludwig-Maximilian University
Munich, Germany
jakob.wankmueller@campus.lmu.de

Adrian Scholl*
M. Sc. Data Science
Ludwig-Maximilian University
Munich, Germany
adrian.scholl@campus.lmu.de

***Abstract*—The blockchain's decentralized nature has fostered financial innovation but also enabled sophisticated fraud, including phishing scams and Ponzi schemes. Traditional fraud detection methods struggle with the scale and complexity of blockchain transactions. This paper presents a graph-based fraud detection approach that uses graph embeddings and traditional machine learning models to classify illicit activity in Ethereum transactions in a computationally cost-effective way. We construct transaction graphs from XBlock data, where nodes represent Ethereum wallets and edges denote transactions. Instead of resource-intensive Graph Neural Networks (GNNs), we employ Graph2Vec and Feather-G embeddings to convert transaction graphs into structured vectors, enabling classification with Random Forest, Support Vector Machines (SVMs), Gradient Boosting (GB) and simple Multi Layer Perceptrons (MLPs). Our results demonstrate that Feather-G embeddings significantly outperform Graph2Vec, preserving critical transaction patterns. Among various classifiers, Gradient Boosting with Feather-G achieves the highest accuracy (93.9%) and F1-score (93.6%), making it the most effective fraud detection model. To facilitate real-world application, we integrate our model into an interactive web tool that enables real-time fraud assessment using the Etherscan API. Our findings highlight the potential of graph embedding-based machine learning in blockchain security while balancing accuracy, interpretability, and computational efficiency.**

***Index Terms*—graph embeddings, fraud detection, blockchain security, ethereum, phishing, ponzi scheme**

I. Introduction

The rapid growth of blockchain technology and cryptocurrencies has transformed the global financial landscape, enabling decentralized applications, digital assets, and borderless transactions. With a market valuation exceeding $1.5 trillion, blockchain networks like Ethereum have become central to the rise of decentralized finance (DeFi), non-fungible tokens (NFTs), and smart contract-based applications. However, as adoption has increased, so have fraudulent activities, with malicious actors exploiting blockchain's pseudonymity and irreversible transactions to carry out sophisticated financial crimes. Among the most prevalent threats are Ponzi schemes and phishing scams, which have led to significant financial losses for users worldwide.

Ponzi schemes in the cryptocurrency space typically disguise themselves as high-yield investment programs, staking platforms, or automated trading bots, luring investors with promises of guaranteed returns. In reality, these schemes rely on a pyramid-like structure, where returns for early investors are paid using the funds of new participants. Once inflows slow down, the system collapses, leaving the majority of investors with heavy losses. Notable examples, such as Bitconnect, have defrauded investors of billions of dollars, yet similar schemes continue to emerge within the rapidly expanding DeFi ecosystem.

Phishing scams, on the other hand, exploit human psychology and social engineering tactics to deceive users into revealing their private keys or signing malicious smart contracts. Attackers often impersonate legitimate exchanges, wallet providers, or well-known figures in the crypto industry, tricking victims into connecting their wallets to fraudulent platforms. Once access is granted, scammers drain funds instantly, making recovery virtually impossible.

Given the increasing complexity and frequency of fraud in blockchain networks, traditional rule-based detection methods have proven inadequate. Graph-based fraud detection has emerged as a powerful alternative, leveraging the structural and relational properties of blockchain transactions to uncover hidden patterns, detect suspicious clusters, and flag illicit activity. By representing transactions as a graph—where accounts act as nodes and interactions as edges—these techniques enable deeper insights into anomalous behaviors and interconnected fraudulent entities.

While Graph Neural Networks (GNNs) have shown potential in blockchain fraud detection, this paper focuses on an alternative approach: graph vector embeddings that transform graph data into fixed-length, tabular data combined with vector-valued machine learning models, such as Random Forests and Support Vector Machines (SVMs). By embedding blockchain transaction graphs into lower-dimensional vector spaces, we can leverage well-established classification algorithms to identify fraudulent behavior. This approach provides interpretability, computational efficiency, and flexibility in detecting suspicious activities within Ethereum's transaction network.

*These authors contributed equally to this work.

II. Related Work

Blockchain fraud detection has been extensively studied, with various approaches leveraging graph-based anomaly detection, machine learning models, and network analytics to identify illicit activities in cryptocurrency transactions. Kang and Buu [1] introduced a disentangled prototypical autoencoder to detect phishing scams in Ethereum transactions, demonstrating the effectiveness of graph anomaly detection in identifying suspicious entities. Similarly, Yu et al. [2] explored graph feature engineering and embeddings for detecting money laundering activities in digital asset transactions, emphasizing the adaptability of machine learning models in blockchain fraud detection.

Graph Neural Networks (GNNs), Graph Convolutional Networks (GCNs), and Convolutional Neural Networks (CNNs) have also been explored in this domain. Ancelotti and Liason [3] provided a review of these models, highlighting their ability to capture structural transaction patterns within blockchain graphs and improve fraud detection and network optimization. While GNNs offer promising results, their high computational costs pose challenges for large-scale blockchain applications. In contrast, Zhao [4] examined Ethereum's transaction network from a temporal graph perspective, analyzing growth rates, update frequencies, and active lifespans of high-degree vertices to detect anomalies. This study introduced forecasting models for predicting community survival, but like many other works, it relied on static graph assumptions rather than continuously evolving transaction data. Kim et al. [5] proposed a graph learning-based approach for detecting phishing accounts on blockchain networks by constructing a heterogeneous transaction graph. Their work demonstrated that modeling multiple types of relationships between blockchain entities enhances detection accuracy and provides valuable insights into the behavior of fraudulent accounts.

Beyond direct fraud detection, graph-based approaches have been applied to Sybil attack detection in decentralized networks. Chandra and Das [6] proposed a graph-based community detection model to identify Sybil nodes in online social networks and blockchain transactions, utilizing anomaly detection and clustering techniques. Additionally, Mahajan et al. [7] explored AI-driven cybersecurity techniques, integrating graph-based manifold learning and density estimation to improve fraud detection in blockchain transactions. Related to the classification approach presented in this paper Kim et al. [8] utilized graph embedding techniques for an identity inference attack on blockchain systems. By harnessing the structural features of blockchain graphs, they effectively inferred user identities, exposing potential weaknesses in privacy-preserving protocols. This study highlights the ability of graph embeddings to capture patterns in a compact representation without compromising their predictive effectiveness.

A. Research Question

While these studies highlight the effectiveness of graph-based machine learning in blockchain fraud detection, challenges remain in computational efficiency and scalability. Most existing models rely on deep learning approaches such as GNNs, which, despite their accuracy, require extensive computational resources.

This work addresses this research gap by leveraging graph vector embeddings combined with traditional machine learning models such as e.g. Random Forests and Support Vector Machines (SVMs). The presented approach offers a more interpretable and computationally efficient alternative, enhancing fraud detection while maintaining scalability in large blockchain networks. In the following we shall examine to what extent this method is suitable to effectively perform fraud detection on blockchain graph transaction data and which embedding algorithms and classifying models yield the most accurate results.

III. Methodology

A. Data

The dataset used in this study is collected from XBlock, a blockchain data-sharing platform designed to facilitate academic research on blockchain analytics. XBlock provides structured datasets for various blockchain applications, enabling reproducible and large-scale analysis. For this work, we utilize the transaction network of phishing nodes dataset from the XBlock transaction-dataset, which includes a mix of verified fraudulent and non-fraudulent Ethereum addresses.

The dataset contains 1,660 verified phishing addresses, 200 verified Ponzi scheme addresses, and 1,700 randomly selected normal addresses with no indications of fraudulent activity. These phishing and Ponzi scheme addresses were sourced from reports published before October 17th, 2019. The transaction network for each address was crawled using the Etherscan API, capturing first-order transaction networks, meaning transactions directly involving the target address and its immediate neighbors. Each transaction record includes the sender and receiver addresses, transaction value, and timestamp.

In addition to the offline dataset used for model training and validation, the same Etherscan API will be utilized during live deployment to retrieve real-time transaction data for any Ethereum wallet under investigation. This allows the fraud detection model to dynamically evaluate new addresses, ensuring the data to remain consistent with the training phase.

Finally, out of the tabular transaction data a graph is created in such a way that each wallet corresponds to a graph node. Edges are then drawn according to the transactions between each pair of sender and receiver addresses. Edge weights are assigned according to the value of ETH tokens transferred.

B. Graph Embeddings

Graph representation learning is a crucial aspect of graph-based machine learning, where the objective is to convert graphs into fixed-length vector representations that preserve structural and feature-based properties. These representations allow to effectively learn graph structures by traditional machine learning methods. Among the various techniques employed for this purpose, Graph2Vec and Feather-G embeddings

are two prominent approaches that sounded promising for our use-case. These methods enable the application of machine learning models which expect tabular data to graphs by transforming them into vectorized forms, facilitating tasks such as classification, clustering, and similarity analysis.

The **Graph2Vec** algorithm [9] draws inspiration from Word2Vec, learning vector representations of entire graphs in an unsupervised manner. It extracts subgraphs, embeds nodes using graph neural networks or walk-based methods, and aggregates these embeddings to form a single vector. The pretrained embeddings are optimized to ensure that structurally similar graphs receive similar representations, making Graph2Vec an effective task-agnostic tool for analyzing various graphical data, e.g. in Bioinformatics and social networks.

Feather G embeddings [10] take a different approach by focusing on pairwise graph similarity. Instead of aggregating subgraphs, it optimizes an embedding space where structurally similar graphs are mapped closer together using contrastive learning. By encoding node and edge-level interactions, Feather G embeddings capture fine-grained graph relationships, making them useful in fraud detection, cybersecurity, and recommender systems.

While Graph2Vec excels at capturing global graph properties, Feather G embeddings are more effective in preserving fine structural distinctions. Since this local sensitivity is more relevant than global features in wallet fraud detection, we expect Feather G embeddings to perform better in this context. But both techniques provide robust fixed-length graph representations that enable the application of different models to our heterogeneous dataset of transaction graphs.

C. Modeling

With our graphical data vectorized and labeled for fraud detection, we tested various machine learning models to identify fraudulent patterns and predict the credibility of unknown wallet addresses. These were the most promising candidates:

Support Vector Machine is a supervised learning algorithm that finds the optimal hyperplane to separate data points into different classes. It does this by maximizing the margin between support vectors, the closest points to the decision boundary. For non-linearly separable data, kernel functions (e.g., RBF, polynomial) project the data into higher-dimensional spaces. The model solves a convex optimization problem, ensuring a globally optimal solution. The kernelized SVM decision rule is given by

$$\hat{f}(x) = \sum_{i=1}^{N} \alpha_i y_i K(x_i, x) + b$$

where $K(x_i, x)$ is the kernel function mapping inputs to a higher-dimensional space, and α_i are the learned coefficients.

Random Forest is an ensemble learning method that builds multiple decision trees using bootstrapped subsets of the training data. Each tree splits nodes based on feature importance (e.g., Gini impurity or entropy) and makes predictions independently. The final classification is determined by majority voting or averaging across all trees, improving robustness and reducing overfitting.

$$\hat{f}(x) = \frac{1}{T} \sum_{t=1}^{T} h_t(x)$$

where $h_t(x)$ is the prediction of the t-th decision tree. Random feature selection at each split enhances generalization and prevents bias toward dominant features.

Gradient Boosting trains decision trees sequentially, with each tree correcting the residual errors of its predecessor. It minimizes a differentiable loss function (e.g., log loss) by iteratively fitting weak learners to negative gradients. The learning rate γ controls step size, balancing accuracy and overfitting, when updating the learner:

$$\hat{f}(x) = f_{m-1}(x) + \gamma h_m(x)$$

where $h_m(x)$ is a weak learner trained on residuals of f_{m-1}. The sequential approach focuses on hard-to-classify instances, often yielding better model fits compared to ensemble methods.

A **Multi-Layer Perceptron** is a fully connected neural network consisting of an input layer, multiple hidden layers, and an output layer. Each neuron applies a weighted sum transformation followed by a non-linear activation function σ (e.g., ReLU) to introduce learning flexibility. The simplest form is given by

$$\hat{f}(x) = \sigma\left(W_L\left(\dots \sigma\left(W_1 x + b_1\right)\dots\right) + b_L\right)$$

Backpropagation adjusts weights by computing gradients through chain rule differentiation and updating them via gradient descent. Dropout and batch normalization help prevent overfitting and stabilize training. It is one of simplest deep learning methods yet flexible and powerful.

A standardized training and evaluation pipeline was used for each model class. This pipeline encorporates embedding calculation, standard scaling, and hyperparameter optimization. Hyperparameter tuning was conducted via grid search cross-validation within a defined parameter space. Model performance was primarily assessed using the F1-score, a metric robust to imbalanced datasets. The optimal hyperparameters, identified through this process, were subsequently used to retrain the model on the complete dataset. While the F1-score served as the primary optimization criterion for the grid search, accuracy, area under the receiver operating characteristic curve (AUC), precision, and recall were also recorded for comprehensive performance evaluation.

IV. Results

The fraud prediction performance of all four classification models was evaluated on an independent hold-out dataset containing 30% of the labeled wallet addresses. Each of these four retrained models was used once with the *Feather-G* and once with the *Graph2Vec* embedding model to classify the held out wallet addresses into fraudulent or non-fraudulent.

The evaluation scores can be interpreted as estimate of the expected prediction performance on previously unseen wallet addresses. Figure 1 displays the evaluation results.

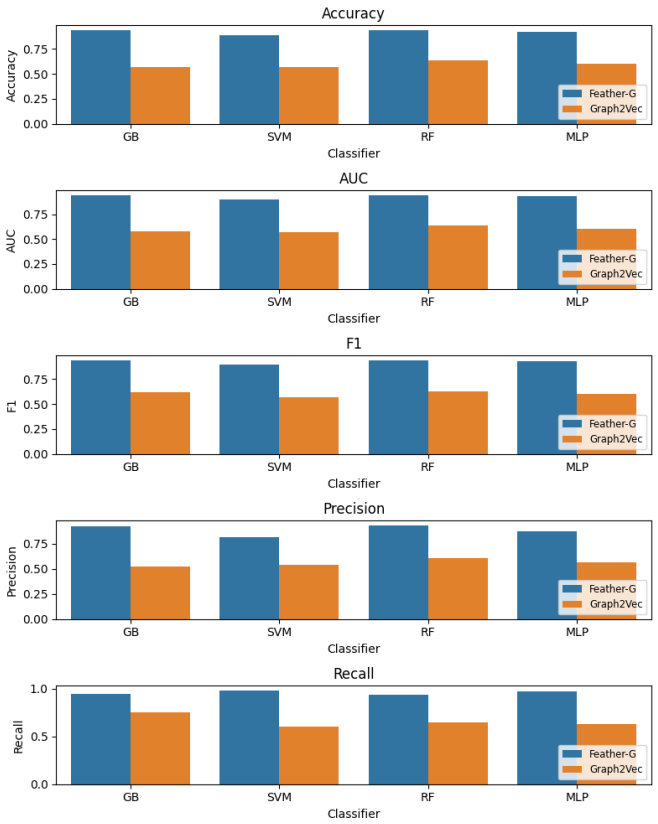


Fig. 1. Evaluation of fraud prediction capabilities of Gradient Boosting (GB), Support Vector Machine (SVM) Random Forest (RF) and Multi Layer Perceptron (MLP) each tested with the Feather-G (blue columns) and the Graph2Vec (orange columns) embedding model.

A. Comparison of Embedding Models

To assess the effectiveness of Feather-G and Graph2Vec embedding models for fraud detection on the Ethereum blockchain, we evaluated their performance across various classification models and metrics. As illustrated in Figure 1, Feather-G consistently outperformed Graph2Vec across all classifiers, demonstrating superior performance in terms of accuracy, AUC, F1-score, precision, and recall.

For instance, when paired with Gradient Boosting, Feather-G achieved an accuracy of 0.939, an AUC of 0.940, and an F1-score of 0.936. In comparison, Graph2Vec with Gradient Boosting yielded significantly lower values, including an accuracy of 0.565, an AUC of 0.577, and an F1-score of 0.618. Similar trends were observed for the Support Vector Machine (SVM), Random Forest, and Multi-Layer Perceptron (MLP) classifiers, confirming the superior representational capability of Feather-G in the context of detecting fraudulent patterns in the Ethereum network.

B. Classifier Performance for Feather-G Embeddings

Given the consistent superiority of Feather-G embeddings, we conducted a detailed comparison of the classifiers using only Feather-G.

Gradient Boosting Classifier (GB) achieved the highest performance among the models, with an accuracy of 0.939, an AUC of 0.940, an F1-score of 0.936, precision of 0.925, and recall of 0.947.

Random Forest (RF) performed competitively with GB, matching the accuracy of 0.939 and F1-score of 0.935, while achieving a precision of 0.934 and a recall of 0.936.

The Support Vector Machine (SVM) showed slightly lower performance, with an accuracy of 0.888, an AUC of 0.894, and an F1-score of 0.891. Notably, it exhibited a high recall of 0.983, indicating its ability to capture most fraudulent cases, albeit at the cost of lower precision (0.815).

Multi-Layer Perceptron (MLP) achieved promising results close to the scores of Gradient Boosting and Random Forest, with an accuracy of 0.925, an AUC of 0.927, an F1-score of 0.924, precision of 0.879, and recall of 0.972. These results demonstrate the potential of MLP as a viable alternative for fraud detection when paired with Feather-G embeddings.

C. Overall Comparison and Best Model Selection

Considering the combined impact of embedding models and classifiers, Feather-G with the Gradient Boosting Classifier emerged as the best-performing combination for detecting fraudulent activities on the Ethereum blockchain. This configuration demonstrated the highest values for accuracy (0.939), AUC (0.940), and F1-score (0.936), alongside a balanced precision (0.925) and recall (0.947). These results suggest that the Gradient Boosting Classifier effectively leverages the nuanced structural information captured by Feather-G embeddings.

The findings indicate the potential of leveraging Feather-G embeddings for blockchain fraud detection tasks, particularly when combined with sophisticated classifiers such as Gradient Boosting. Future work may explore optimizing the parameters of other classifiers and embedding models to further enhance detection performance.

TABLE I
PERFORMANCE EVALUATION OF FRAUD DETECTION MODELS USING GRAPH2VEC AND FEATHER-G EMBEDDINGS

Model	Accuracy	AUC	F1-Score	Precision	Recall
Graph2Vec+GB	0.565	0.577	**0.618**	0.525	**0.752**
Graph2Vec+SVM	0.571	0.573	0.569	**0.538**	0.604
Graph2Vec+RF	**0.640**	**0.640**	0.626	0.609	0.644
Graph2Vec+MLP	0.603	0.605	0.598	0.569	0.631
Feather-G+GB	**0.939**	**0.940**	**0.936**	0.925	0.947
Feather-G+SVM	0.888	0.894	0.891	0.815	**0.983**
Feather-G+RF	**0.939**	0.939	0.935	**0.934**	0.936
Feather-G+MLP	0.925	0.927	0.924	0.879	0.972

D. Distribution of scores on training set

While the aforementioned scores provide valuable insights into the accuracy of sample classifications, they are based on discrete (hard) labels. However, our model generates a

continuous (soft) score ranging from 0 to 1, representing the probability that a given wallet is fraudulent. Examining these probabilistic scores offers a measure of confidence in each prediction, making it informative to analyze the distribution of fraud scores across the entire training dataset.

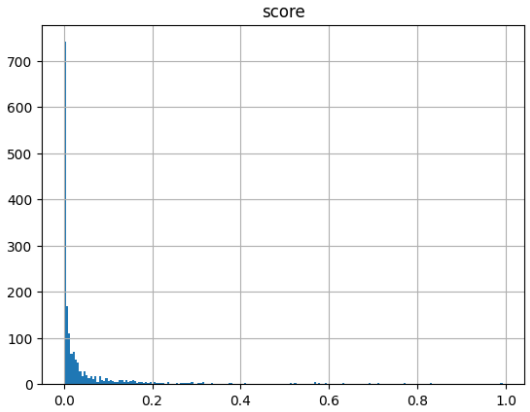


Fig. 2. Histogram of soft scores across normal training addresses

To provide an intuitive visual representation of these distributions, we generated histograms for each wallet type (normal, phishing, and Ponzi). These histograms illustrate the frequency of each fraud score within the 0 to 1 range when performing inference using the best model, which was retrained on the entire training dataset. The distributions were captured using 200 bins to ensure a detailed depiction of the score distribution.

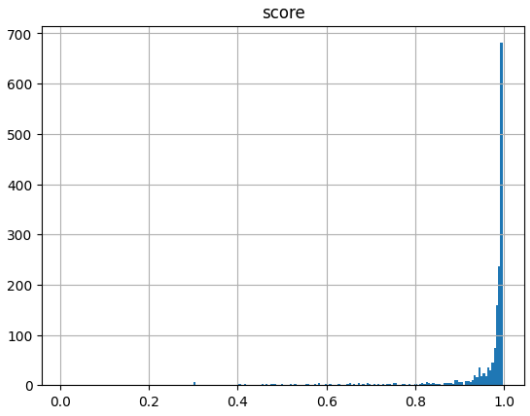


Fig. 3. Histogram of soft scores across phishing training addresses

Figures 2 and 3 present highly promising results. The distribution of fraud scores for non-fraudulent wallet addresses is predominantly concentrated within the bottom quartile, with a 25th percentile score of 0.001372. Conversely, phishing wallets exhibit a distribution primarily within the top quartile, with a 75th percentile score of 0.994401. Furthermore, the presence of outliers is minimal. Notably, the highest-frequency bin contains approximately 700 entries in both cases, representing nearly half of the training dataset.

For wallets involved in Ponzi schemes, the sample size in our dataset is significantly smaller, resulting in a fraud

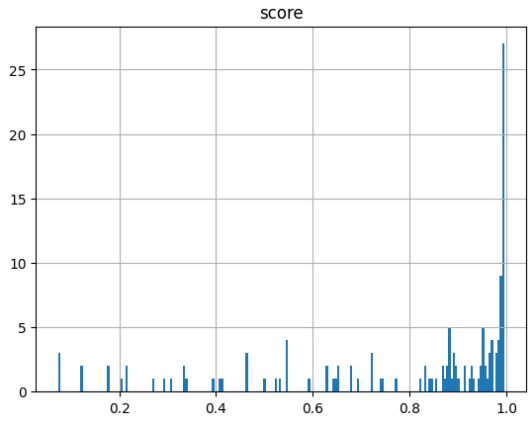


Fig. 4. Histogram of soft scores across ponzi training addresses

probability histogram that appears noticeably noisier, with frequent outliers distributed across the entire range from 0 to 1. Nevertheless, the majority of these wallets are correctly assigned a high fraud score, as evidenced by a pronounced spike in the highest bin. The 75th percentile score is 0.987007, only marginally lower than that of phishing wallets. However, the 50th percentile score is considerably lower at 0.900520 (compared to 0.989226 for phishing wallets), indicating that approximately 10% of Ponzi scheme wallets are misclassified as non-fraudulent.

V. Implementation

To make the results of this research accessible, we integrated our fraud detection framework with the best graph embedding and fraud prediction model into a publicly accessible fraud detection tool, available at https://ethxpose.vercel.app/. The web application is composed of a backend API built using *FastAPI* in Python and an interactive frontend web interface developed with *Next.js*.

A. FastAPI Backend

The core of this web tool is a FastAPI backend that provides the *classify* API endpoint. This endpoint allows users to send a POST request to the server with a wallet address for classification. When a POST request is made to the `/api/py/classify` endpoint, the backend executes a structured workflow that processes wallet data, generates a classification, and returns a comprehensive response:

1) The API accepts a JSON payload containing two key fields: the Ethereum wallet address to be classified and the model name for classification (defaulting to the best model `Graph2Vec_RF`). The input JSON is validated via a *Pydantic* data model.
2) The transaction data for the specified wallet address is retrieved from the Etherscan API. The backend uses the `fetch_transactions` function to send API requests with the wallet address and block range parameters. If successful, the response is converted into a

Pandas DataFrame. Transactions with a value of zero are excluded to only retain meaningful data.

3) The filtered transaction data is transformed into a graph structure using *NetworkX*. In this graph nodes represent wallet addresses and edges denote transactions between these addresses, with attributes such as the transaction value and the transaction timestamp. This graph structure captures the transactional behavior of the wallet requested for classification.
4) Then the graph embeddings are generated to enable the machine learning-based classification. The `get_graph_embeddings` function fits the embedding model to the graph and extracts the numerical representations required for inference.
5) The already trained scikit-learn pipeline containing a standard scaler and the tuned machine model with its best hyperparameters is loaded to predict the probability of fraudulent activity. For the prediction the graph embedding of the transaction graph is passed scikit-learn pipeline, which then outputs a fraud probability using the `predict_proba` method.
6) In the last step, the predicted fraud probability and the transaction graph have to be returned to the front-end for the visualization of the graph and the display of the classification result. To ensure a stable data transfer, the data are encoded in a JSON representation and then send to the client. In case of any errors, the backend returns an appropriate HTTP status code with an error message.

This workflow enables the backend service to provide a robust fraud detection service via a scalable RESTful API. The entire backend application is served using *Uvicorn*, a fast ASGI server.

B. Frontend with Next.js and C3 Graph Visualization

The frontend enables users to input Ethereum wallet addresses, validate the address format, send requests to the backend for fraud classification, and visualize Ethereum transaction sub networks. It is implemented using *Next.js*, providing a seamless and dynamic user experience.

To communicate with the backend, *Axios* is used to make HTTP requests. Specifically, when a user inputs a wallet address, the frontend sends an asynchronous POST request to the backend API, requesting a fraud classification.

For a dynamic and interactive data visualization of the Ethereum sub network around the classified wallet address, *C3.js* is employed. *C3.js* renders transaction graphs by initializing the chart container, configuring the graph structure based on transaction data, and rendering nodes and edges, where nodes represent addresses and edges represent transactions. The visualization is designed to be interactive, offering users better insights into the transaction network. As for the backend, error handling is an integral part of the frontend design. The system handles errors by displaying appropriate messages for issues such as invalid wallet addresses and backend communication failures.

The implementation of the Ethereum fraud detection platform combines a robust backend API and an interactive frontend, enabling users to analyze wallet transactions and identify potential fraud efficiently. By leveraging modern web and data visualization technologies, the platform provides an engaging and informative user experience.

VI. Discussion

Our evaluation results suggest that the models achieve a very strong fit for the specific task of Ethereum fraud detection, allowing for highly confident predictions. The best-performing configurations, particularly those using Feather-G embeddings, consistently show high accuracy, F1-scores, and recall, indicating the model's ability to detect fraudulent activity with minimal false negatives. This strong performance suggests that our approach is well-suited for identifying suspicious wallets and can be reliably applied in real-world scenarios.

The tree-based models performed best overall due to their great adaptability, Gradient Boosting (GB) however delivered only a minimal score improvement over the simpler Random Forest (RF) which is also generally less prone to overfitting. Although the Multi-Layer Perceptron (MLP) is a powerful model, it requires more extensive hyperparameter tuning than traditional machine learning methods. Given that Random Forest and Gradient Boosting performed just as well while being more computationally efficient, deep learning approaches appear unnecessary for this use case. However, while all classifiers performed well with Feather-G embeddings, the choice of embedding method proved to be far more influential than the choice of machine learning model. The Graph2Vec-based models performed significantly worse across all metrics, while the Feather-G-based models—regardless of whether they used Random Forest, Gradient Boosting, or SVM—produced relatively similar results. This highlights the importance of selecting the right graph representation method over fine-tuning subsequent classification model as a key takeaway.

Additionally, while our dataset is balanced between fraudulent and non-fraudulent addresses, phishing scams outnumber Ponzi schemes within the fraud category. Since we train a single binary classifier, the model is naturally better at identifying phishing patterns, leading to a lower detection rate for Ponzi schemes. This dataset imbalance may limit the generalizability of our model across different types of fraud. These challenges point to broader limitations in our approach, which we will discuss in the next section.

A. Limitations

A key limitation of our approach is the scale of the training dataset relative to the vastness of the Ethereum blockchain. While the dataset is not small, it still represents only a tiny fraction of all wallets, meaning the model's ability to generalize depends on the assumption that both fraudulent and non-fraudulent training data are representative of the overall population. This assumption cannot be fully guaranteed, as fraudulent behaviors evolve and may differ significantly outside the observed dataset. Additionally, there is an inherent ad-

verse selection bias in the collection of fraudulent data. Scam databases primarily contain addresses that have already been identified and reported, but the most sophisticated fraudsters are often the hardest to detect. As a result, our dataset is likely skewed toward easier-to-spot scams, potentially limiting the model's ability to recognize more subtle or emerging fraud patterns.

One other major limitation of graph-based fraud detection in Ethereum transactions arises from the sheer volume of interactions associated with certain wallets. High-activity nodes, such as centralized exchanges, DeFi platforms, or frequently used scam wallets, can have an extremely large first-order neighborhood consisting of thousands or even millions of direct connections. This makes even basic first-order graph analysis computationally expensive, as the adjacency matrix grows rapidly. Consequently, extending the analysis to higher-order neighborhoods—which could provide deeper insights into transactional relationships—becomes nearly infeasible due to exponential increases in data size and complexity. Like many traditional machine learning models, our approach struggles to handle these dense subgraphs effectively, leading to scalability issues and the need for extensive pruning or sampling, which may remove crucial fraud-related patterns.

Another challenge is that fraudsters can intentionally evade detection by avoiding clear fraudulent patterns in the transaction graph. Rather than reusing addresses, scammers frequently generate new wallet addresses for each fraudulent transaction, making it difficult to establish persistent relationships between accounts. Since blockchain transactions are pseudonymous and permissionless, bad actors can automate the creation of fresh addresses, preventing graph-based detection models from identifying recurrent fraudulent clusters. This strategy undermines the effectiveness of graph-based anomaly detection, as models typically rely on recognizing suspicious structures and behavioral patterns over time. Consequently, fraud detection systems should integrate methods beyond the blockchain, like tracking cross-wallet activity to remain effective against these evolving evasion tactics.

B. Future Work

Future work could compare the current graph embedding approach with Graph Neural Networks (GNNs) to evaluate trade-offs in accuracy, speed, and interpretability. While embeddings are efficient and work well with traditional machine learning models, GNNs may better capture complex fraud patterns. Testing models Temporal Graph Neural Networks (TGNNs) to track evolving fraud patterns over time could enhance fraud identifiability.

Another key direction is expanding the dataset to include multi-hop transactions, cross-blockchain activity, and synthetic fraud cases which could also improve detection accuracy. Integrating external data sources, such as social media and regulatory watchlists, may also provide a broader view of fraudulent behavior. Finally, adding a continuous update and feedback system—where flagged addresses are monitored, and model performance is regularly reassessed—would help the system stay effective against new fraud tactics.

REFERENCES

[1] J. Kang and S.J. Buu, "Graph Anomaly Detection with Disentangled Prototypical Autoencoder for Phishing Scam Detection in Cryptocurrency Transactions," *IEEE Access*, 2024.

[2] Q. Yu, Z. Ke, G. Xiong, Y. Cheng, and X. Guo, "Identifying Money Laundering Risks in Digital Asset Transactions Based on AI Algorithms," *EasyChair Preprint*, 2025.

[3] A. Ancelotti and C. Liason, "Review of Blockchain Application with Graph Neural Networks, Graph Convolutional Networks, and Convolutional Neural Networks," *arXiv preprint*, 2024.

[4] L. Zhao, "Enabling Advanced Analytics for Industry Applications Using Machine Learning and Graph Mining," Doctoral Thesis, Nanyang Technological University, 2024.

[5] J. Kim, S. Lee, Y. Kim, S. Ahn, and S. Cho, "Graph Learning-Based Blockchain Phishing Account Detection with a Heterogeneous Transaction Graph," *Sensors*, vol. 23, no. 1, p. 463, 2023, doi: 10.3390/s23010463.

[6] S. Chandra and P. Das, "A Graph-Based Approach for Detecting Sybil Nodes in Online Social Networks," *Proceedings of the 15th International Conference on AI Security*, IEEE, 2024.

[7] S. Mahajan, M. Khurana, and V.V. Estrela, *Applying Artificial Intelligence in Cybersecurity Analytics and Cyber Threat Detection*, Google Books, 2024.

[8] J. Kim, S. Lee, Y. Kim, and S. Cho, "A Graph Embedding-based Identity Inference Attack on Blockchain Systems," *Proceedings of the 2022 International Conference on Electronics, Information, and Communication (ICEIC)*, pp. 1-3, 2022, doi: 10.1109/ICEIC54506.2022.9748281.

[9] A. Narayanan, B. Chandrasekaran, and A. Choudhary, "graph2vec: Learning Distributed Representations of Graphs," *arXiv preprint*, 2017.

[10] P. Orsini, M. Frasca, A. Bertoni, and G. Boracchi, "Characteristic Functions on Graphs: Birds of a Feather, from Statistical Descriptors to Parametric Models," *arXiv preprint*, 2023.